Chapter 8 — Prompting vs Fine-Tuning vs RAG, and Evaluating LLM Systems

Last reviewed: September 2026. Tooling and model names move quickly; the decision logic, the LoRA arithmetic and the evaluation methods in this chapter do not.

A general-purpose model rarely fits a product out of the box: it may lack your facts, miss your format or tone, or cost too much at volume. Three tools close the gap — **prompting**, **retrieval (RAG)** (looking up relevant documents and adding them to the prompt) and **fine-tuning** (continuing to train the model on your examples) — and they fix *different* problems. The second half answers whether any of them worked: **how do you evaluate a system whose correct answers can be written a thousand ways?**

Covered: a decision framework; full vs parameter-efficient fine-tuning (LoRA, QLoRA) with the maths and memory numbers; tuning data (SFT, preferences, distillation) and when tuning hurts; continued pretraining and training from scratch; the evaluation pyramid (programmatic checks, LLM-as-judge and its biases, human evaluation); golden sets, CI regression testing, metrics (EM, F1, BLEU/ROUGE and their limits, pass@k) and statistical care; online evaluation (feedback, proxies, outcomes, A/B tests, canaries); guardrail evals and tooling.

Runnable code in `code/`: a LoRA parameter and memory calculator with a tiny LoRA layer trained in PyTorch, and an evaluation harness scoring fake models on a golden set (a fixed collection of test inputs with known good answers; §8.5). Prerequisites: Chapters 4 (how LLMs are trained), 5 (prompting) and 7 (RAG); gradient descent and loss functions from the ML Handbook.

8.1 The adaptation question: three tools for three problems

What it is

Adapting a model means making a general model behave well for *your* task; interviewers ask directly ("would you fine-tune here?") or wait to see if you raise it. The three tools are not a ladder from basic to advanced; each addresses a different gap. Rule of thumb: **retrieval hands the model facts while it answers; fine-tuning reshapes how it behaves by updating its weights** — a guideline, not a wall.

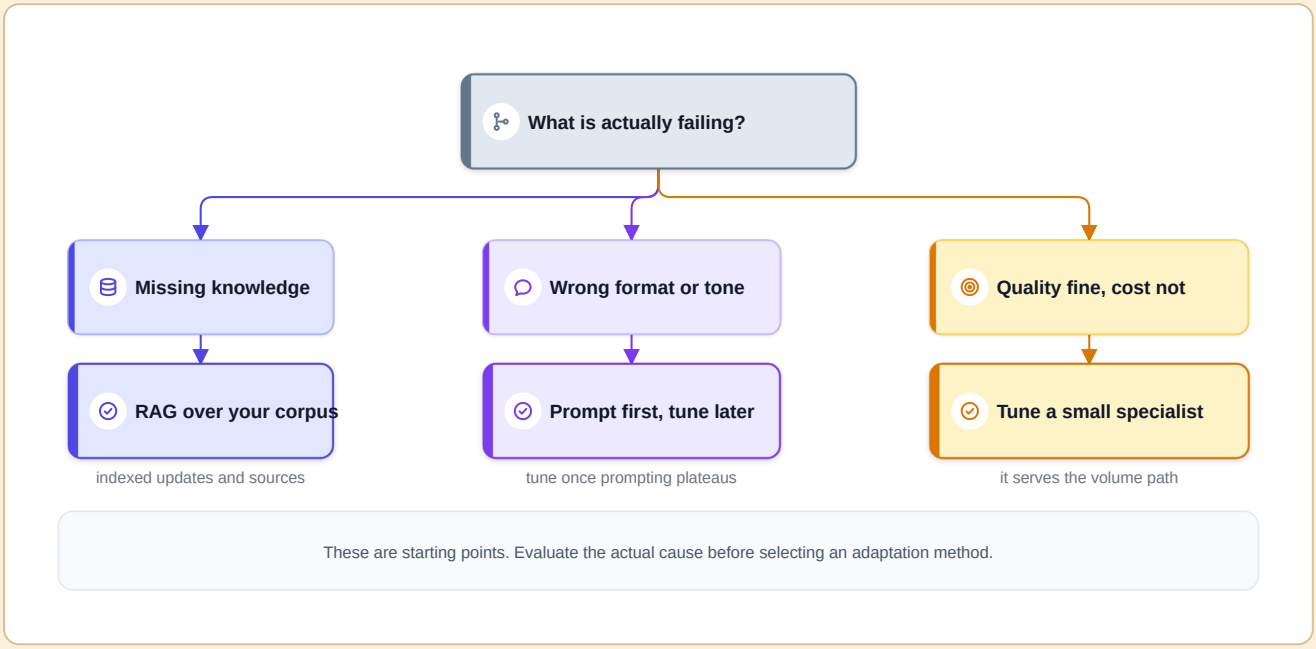


Figure — Start from the failure you observe, not from the fanciest tool (course diagram).

The three tools

1. Prompting (with few-shot examples). Each request carries the task, a persona, the output shape and a few worked input→output pairs (**few-shot examples**).

- Weights untouched: a change deploys like a config edit — though it must still pass your evals and roll out gradually.
- Instructions and examples cost context tokens on every call; long or contradictory prompts get less reliable.
- A prompt can *supply* facts, but they don't persist in the weights.
- Start here: clear prompt, representative examples, measure what still fails, *then* decide on retrieval or training.

2. RAG. Fetch relevant facts (documents, rows, user records) at request time; the model answers **grounded** in them (Chapter 7).

- The tool for **knowledge**: private, large or fast-changing.
- Updates appear once documents are ingested, indexed and caches invalidated — easier than retraining, not instant.
- Sources can be shown; **access control is enforced per user at retrieval time**.
- Costs: a retrieval subsystem to build and evaluate, longer prompts, answer quality capped by retrieval quality.

3. Fine-tuning. Continue training an existing model on curated examples (often input→output pairs).

- The tool for **behaviour**: reliable formats, a house voice, domain reasoning patterns or task skills.
- Can shorten prompts and let a **smaller, cheaper model** handle a narrow task — if evaluation confirms quality.
- Costs: data curation, training, evaluation, deployment and versioning; iterations take hours to days.
- **Poor at frequently changing or access-controlled facts**: models memorise unevenly, every update needs retraining, and weights can't check who is asking. For an internal wiki, start with permission-aware retrieval.

The decision framework

Start from **the failure you actually observe**, not the most impressive technique.

Observed problem	Likely cause to check first	Start with	Escalate to
Wrong or outdated facts; "I don't know our policy"	Missing or stale evidence; poor retrieval	**RAG** (fix sources and retrieval)	Better chunking, hybrid + rerank (Chapter 7); tuning for evidence use
Answers private/user-specific data	Needs per-user data and permissions	**RAG** with access control	— (never weights)
Wrong format, schema or length	Unclear instructions	**Prompt** + examples, structured outputs, validation	Fine-tune if errors persist
Wrong tone or style	Underspecified voice	**Prompt** with examples	Fine-tune for consistent voice
Can't do the task well at all	Model capability	Better prompt, stronger model	Fine-tune on task data
Quality fine, but too expensive or slow at volume	Model size	Try an existing smaller model	Fine-tune (or distil into, i.e. train a small model to copy a big one) a small specialist
Tuned model invents policy details	Evidence path, not training	Check that current policy is available, retrieved and used; add grounding checks	More tuning alone won't keep policies current

- **Diagnose before choosing.** "Wrong facts" can mean stale sources, bad retrieval or the model ignoring good evidence — three fixes. "Bad formatting" may vanish with sharper instructions, a JSON schema or constrained decoding (only schema-valid tokens allowed) — or, failing those, tuning.

- **Combine when benefits justify complexity.** Common: **retrieval supplies policy facts while a small tuned model supplies voice and structure**, wrapped in a prompt. When cost drives it, weigh an off-the-shelf small model against a tuned one, counting training and upkeep.

What about training your own model?

- **Pretraining from scratch** a foundation model needs vast data, compute and expertise — outside almost every product design question; if proposed, justify why no existing model works.
- **Continued pretraining** — more unsupervised training of an existing model on a large domain corpus (e.g. Code Llama was Llama 2 continued on code) — can improve domain fluency but needs its own quality, cost and data-governance case. More training isn't automatically the answer to missing or changing knowledge.

Common mistakes

- Fine-tuning to "teach the model our documents" when they change monthly and have permissions.
- Tuning before a strong prompt and an evaluation set exist — you can't tell whether it helped.
- Treating the three as mutually exclusive.

Real-world examples

Support assistants pair RAG over help-centre articles with prompts or small tuned models for tone; coding assistants fine-tune on code and retrieve over the user's repository; many companies fine-tune small open models (e.g. LoRA via Hugging Face PEFT, §8.2) for high-volume classification or extraction to cut cost. Hosted fine-tuning (OpenAI's fine-tuning API, Amazon Bedrock, Google Vertex AI) tunes selected models without managing GPUs; which models are tunable changes often.

In the interview

> **30-second answer:** "First a solid prompt and an eval set, so I can see what actually fails. Failures about recent or private facts → retrieval with access checks. Format, tone or task errors that survive better prompting and output constraints → consider tuning; if cost is the issue, benchmark a tuned small model against an untuned small one. The approaches stack, and none replaces authorization, output validation or production monitoring."

- *Where would training examples come from?* Logs you may use (scrubbed of personal data, within retention rules), well-rated conversations, reviewer-corrected answers, and synthetic pairs from a stronger model screened by people — today's traffic becomes tomorrow's training set.
- *Does RAG eliminate hallucination?* No; it reduces it. Grounding instructions, citations and claim checks complete it (Chapter 7.8).
- *Full fine-tuning or LoRA?* LoRA freezes base weights and trains small low-rank updates, cutting training memory and storage; compare quality with full fine-tuning for your task. The decision logic is the same either way.

Self-check

1. Outputs correct but JSON often malformed — first step? → Clear instructions + examples, structured outputs and validation; fine-tune only if that plateaus.
2. Why not fine-tune on per-user confidential data? → Weights can't enforce access control, and updates need retraining.

TL;DR

Prompting is the baseline; RAG adds knowledge (fresh, private, permissioned); fine-tuning changes behaviour (format, tone, skill, cost). Diagnose the failure, start simple, combine when justified, never tune without evals.

8.2 Fine-tuning: full vs parameter-efficient

What it is

Full fine-tuning updates every weight. **Parameter-efficient fine-tuning (PEFT)** freezes the original weights and trains a few new or selected parameters. The most popular PEFT method is **LoRA** (Low-Rank Adaptation; Hu et al., 2021); **QLoRA** (Dettmers et al., 2023) runs LoRA on a 4-bit-quantised frozen model (4 bits per weight instead of 16: a little precision for much less memory).

Why it matters: the memory wall

Training with the Adam optimiser in mixed precision needs per parameter roughly 2 bytes (bf16 weight) + 2 (gradient) + 4 (fp32 master copy) + 4 + 4 (Adam's two moment estimates) = **16 bytes**, before activations.

> **Key term — what training memory holds:** An *optimiser* updates the weights; **Adam** keeps two running averages (*moment estimates*) of each weight's recent gradients. A *gradient* says how much and which way each weight should change. *Mixed precision* does most maths in 16-bit (**bf16**) while keeping a 32-bit (**fp32**) *master copy* for accurate updates. *Activations* are intermediate forward-pass values saved for computing gradients; they grow with batch size and sequence length. For a 7-billion-parameter model that is ~108 GB — more than one 80 GB GPU — and every fine-tuned variant is another full 13.5 GB checkpoint to store and serve.

How LoRA works

Hypothesis (supported by the LoRA paper): the *change* a task needs, ΔW, has **low intrinsic rank** (few truly independent directions), so it needs far fewer numbers than the full matrix.

1. Take a frozen weight matrix W_0 of shape d × k (e.g. a 4,096 × 4,096 attention projection; Chapter 3).
2. Add a trainable $\Delta W = B \cdot A$, with **A** r × k, **B** d × r and **rank r** small (4–64). The layer computes $h = W_0 x + (\alpha/r) \cdot B \cdot A \cdot x$.
3. Initialise A small and random, **B to zeros**, so B·A = 0 and training starts exactly at the original model.
4. Train only A and B. **α** (lora_alpha) is a scaling constant; dividing by r keeps the update's size comparable across r.
5. Afterwards keep the small adapter separate (swap per customer or task on one base model) or **merge**: $W' = W_0 + (\alpha/r) \cdot B \cdot A$ — no extra inference latency (see System Design Fundamentals).

Parameter arithmetic for one 4,096 × 4,096 matrix: full = 16,777,216; LoRA r = 8 trains 8 × (4,096 + 4,096) = **65,536** — 0.39%. On GPT-3 175B the LoRA paper reports ~10,000× fewer trainable parameters and ~3× less GPU memory than full fine-tuning, with comparable benchmark quality.

Which matrices? The paper adapted attention query and value projections; practice now often targets all linear layers (attention and MLP), usually better quality at modest cost.

Choosing r. Start around 8–16; raise it if the task is far from base behaviour and data supports it. Too small underfits; too large wastes memory and can overfit small datasets.

How QLoRA works

1. Quantise the frozen base model to 4 bits with the **NF4** (4-bit NormalFloat) data type, designed for normally distributed weights.
2. **Double quantisation**: also quantise the per-block scaling constants.
3. **Paged optimisers** move optimiser state between GPU and CPU memory to survive spikes.
4. Train LoRA adapters in 16-bit on top; gradients flow through the dequantised weights.

QLoRA fine-tuned a 65B model on a single 48 GB GPU while matching 16-bit fine-tuning on its benchmarks.

Code: parameter counts, memory, and a real LoRA layer (runs offline)

`code/lora_calculator.py` uses Llama-2-7B shapes (hidden size 4,096, 32 layers, MLP size 11,008, vocabulary 32,000).

```
def lora_params(c, r, targets):
    shapes = {"q": (c["d"], c["d"]), "k": (c["d"], c["d"]), "v": (c["d"], c["d"]),
              "o": (c["d"], c["d"]), "gate": (c["d"], c["ffn"]), "up": (c["d"], c["ffn"]),
              "down": (c["ffn"], c["d"])}
    # each adapted matrix W (d_in x d_out) gets A (r x d_in) and B (d_out x r)
    return c["layers"] * sum(r * (shapes[t][0] + shapes[t][1]) for t in targets)

class LoRALinear(nn.Module):
    def __init__(self, base: nn.Linear, r: int, alpha: float):
        super().__init__()
        self.base = base
        for p in self.base.parameters():
            p.requires_grad = False                                    # frozen W0
        self.A = nn.Parameter(torch.randn(r, base.in_features) * 0.01)  # small random
        self.B = nn.Parameter(torch.zeros(base.out_features, r))        # zero: starts at W0
        self.scale = alpha / r

    def forward(self, x):
        return self.base(x) + self.scale * (x @ self.A.T @ self.B.T)

    def merged(self):
        return self.base.weight + self.scale * self.B @ self.A          # W0 + (alpha/r)·B·A
```

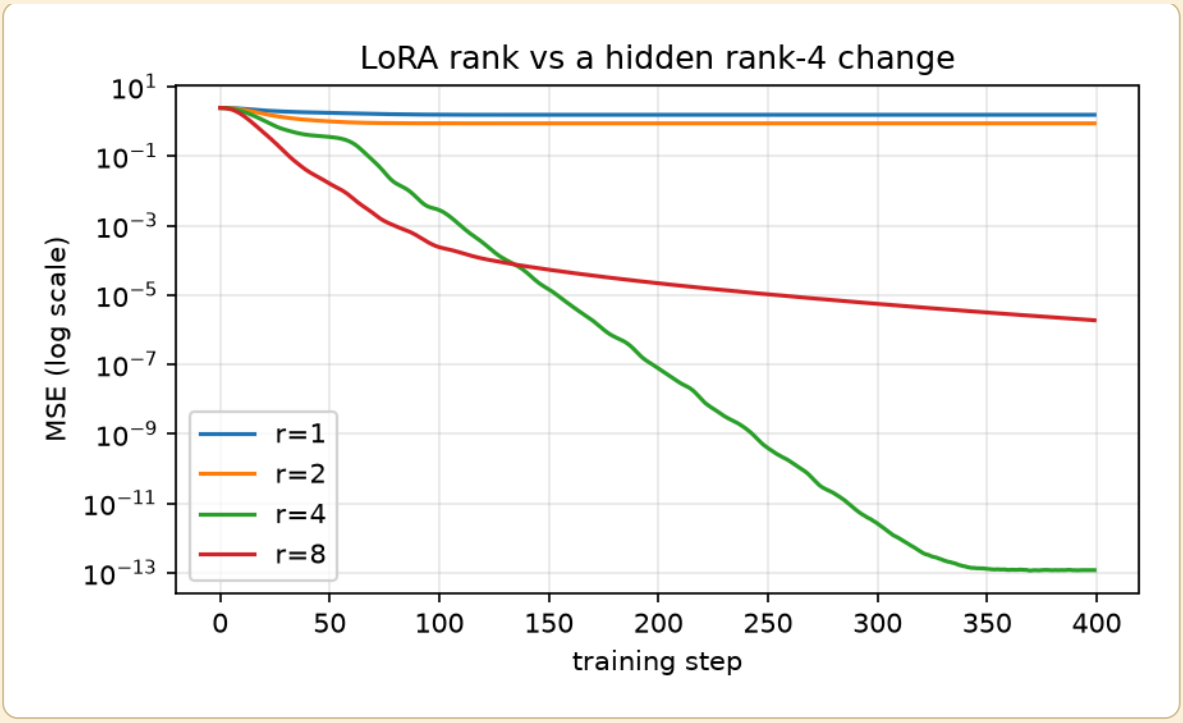
Output:

```
Base model parameters: 6.74 B

setup                                trainable  % of base
LoRA r=8 on q,v (LoRA paper style)       4.2 M      0.06%
LoRA r=16 on attention                  16.8 M      0.25%
LoRA r=16 on all linear                 40.0 M      0.59%
LoRA r=64 on all linear                159.9 M      2.37%

Rough GPU memory for weights + gradients + Adam state (no activations):
  full fine-tune (16 bytes/param)       107.8 GB
  LoRA, bf16 frozen base + adapters      14.1 GB
  QLoRA, 4-bit frozen base + adapters     4.0 GB
  one saved adapter (bf16)               80.0 MB  vs full checkpoint 13.5 GB

Tiny LoRA layer (64x64 frozen, hidden rank-4 change):
  r=1: trainable  128 of 4288 total params, final MSE 1.56e+00
  r=2: trainable  256 of 4416 total params, final MSE 8.82e-01
  r=4: trainable  512 of 4672 total params, final MSE 1.18e-13
  r=8: trainable 1024 of 5184 total params, final MSE 1.88e-06
  merged W' = W0 + (alpha/r)BA gives same outputs: max diff 4.3e-06
saved img/lora_rank_loss.png
```



The 7B attention-only r = 8 adapter trains 0.06% of weights; all-linear r = 64 under 2.5%. Memory falls from ~108 GB to ~14 GB (LoRA) and ~4 GB (QLoRA) for weights and optimiser state, activations extra. In the toy layer the true change has rank 4: ranks 1 and 2 can't represent it (loss plateaus), rank 4 fits it exactly with 11% of the layer's parameters, and merging reproduces adapter outputs to floating-point error — why merged LoRA adds no inference cost.

The same with Hugging Face PEFT (not run here — needs a model download and a GPU)

```
from transformers import AutoModelForCausalLM
from peft import LoraConfig, get_peft_model

model = AutoModelForCausalLM.from_pretrained("<base-model-id>")
config = LoraConfig(r=16, lora_alpha=32, lora_dropout=0.05,
                    target_modules=["q_proj", "k_proj", "v_proj", "o_proj"],
                    task_type="CAUSAL_LM")
model = get_peft_model(model, config)
model.print_trainable_parameters()      # a fraction of a percent
# ... train with your usual loop or the transformers Trainer ...
model = model.merge_and_unload()        # optional: fold adapters into the weights
```

For QLoRA, load the base in 4-bit (bitsandbytes: `BitsAndBytesConfig` with `load_in_4bit=True` and `bnb_4bit_quant_type="nf4"`) before attaching adapters. Module names (`q_proj`…) depend on the architecture.

Trade-offs

	Full fine-tuning	LoRA	QLoRA
Trainable params	100%	~0.1–3%	~0.1–3%
Training memory (7B, excl. activations)	~108 GB	~14 GB	~4 GB
Quality ceiling	Highest in principle	Often close on narrow tasks	Close to LoRA; slightly slower (dequantisation)
Artefact per task	Full checkpoint (GBs)	Adapter (MBs), swappable	Adapter
Forgetting risk	Higher	Lower (base frozen)	Lower

Other PEFT methods — adapter layers (Houlsby et al., 2019), prefix tuning, prompt tuning (learned soft-prompt vectors), IA³, DoRA — exist, but LoRA-style adapters dominate.

Common mistakes

- A huge rank "to be safe" on a few hundred examples (overfitting).
- Forgetting activation memory when sizing GPUs.
- Evaluating the adapter only on the new task, missing regressions elsewhere.
- Comparing to an unpinned base model (its version may change under you).

In the interview

30-second answer: "LoRA freezes W and learns ΔW = BA with rank r — A is r×k, B is d×r, B starts at zero so training starts from the base model. For a 4096×4096 matrix at r=8 that's 65K trainable parameters instead of 16.8M. Full fine-tuning a 7B model needs ~16 bytes per parameter, over 100 GB before activations; LoRA needs roughly the bf16 weights plus tiny adapters, and QLoRA quantises the frozen base to 4-bit NF4 for single-GPU budgets. Adapters are megabytes, swappable per task, and mergeable for zero added latency."

- *Why initialise B to zero?* ΔW = 0 at step 0, so training starts exactly at the base model and moves away smoothly.
- *What does α do?* Scales the update; α/r keeps its magnitude comparable across r, so learning rates transfer.
- *When prefer full fine-tuning?* Large distribution shift, abundant data and compute, and evidence that adapters plateau below target.

Self-check

1. LoRA params for a 4,096 × 11,008 matrix at r = 16? → 16 × (4,096 + 11,008) = 241,664.
2. What does QLoRA quantise? → The frozen base weights (4-bit NF4); adapters train in 16-bit.
3. Full fine-tuning memory for 13B at 16 bytes/param? → ~208 GB before activations.

TL;DR

LoRA trains a low-rank update BA beside frozen weights: tiny, swappable, mergeable. QLoRA adds a 4-bit base for single-GPU training. Size full fine-tuning at ~16 bytes/param.

8.3 Tuning data: instructions, preferences and distillation

What it is

The data decides what fine-tuning teaches:

Stage	Data format	What it teaches	Example
Supervised fine-tuning (SFT) / instruction tuning	(instruction, ideal response) pairs, or multi-turn chats	Follow instructions, a format, a task	FLAN; the SFT step of InstructGPT
Preference tuning	(prompt, chosen response, rejected response)	Which of two answers people prefer: helpfulness, safety, style	RLHF in InstructGPT; DPO
Distillation	A strong **teacher** model's outputs (or probabilities) on many inputs	Make a small **student** imitate the teacher	Hinton et al. (2015); Stanford Alpaca (52K instructions generated with an OpenAI model)

How it works

SFT is next-token training with loss usually only on response tokens. Quality beats quantity: LIMA (Zhou et al., 2023) got a strong instruction follower from ~1,000 curated examples on a good base

model. Sources: permitted product logs (privacy and retention rules), top-rated conversations, human-corrected outputs from review queues, and synthetic examples from a frontier model filtered by people.

Preference tuning (mechanics in Chapter 4 §4.6): **RLHF** (Ouyang et al., 2022) trains a **reward model** on human rankings and optimises the LLM against it with RL (PPO), penalising drift from the SFT model; **DPO** (Rafailov et al., 2023) skips the reward model and RL loop, training directly on chosen/rejected pairs with a classification-style loss — simpler and now common.

Distillation classically trains the student on the teacher's softened probability distribution ("soft targets"), which carries more information than hard labels. With LLMs it often means generating a large synthetic dataset with the teacher and running SFT on the student — how many small, cheap production models are made. Check the teacher's terms of use; some prohibit building competing models from outputs.

When fine-tuning hurts

- **Catastrophic forgetting**: gains on the new task, losses on untested general abilities.
- **Safety erosion**: Qi et al. (2023) showed fine-tuning aligned models — even on small or benign datasets — can weaken safety behaviour. Re-run safety evals after every tune.
- **More hallucination from new facts**: teaching facts the base model didn't know can encourage confident guessing (Gekhman et al., 2024: such examples are learned slowly and increase hallucination tendency).
- **Stale knowledge baked in**: facts in weights go out of date; retraining is slow.
- **Overfitting to a narrow format**: great in-distribution, brittle elsewhere.
- **Operational drag**: every base-model upgrade means re-tuning and re-evaluating.

Common mistakes

- Training on raw logs without removing personal data or bad answers.
- No held-out test set separate from tuning data.
- Mixing train and evaluation examples (leakage makes results look great).

Self-check

1. How does DPO differ from RLHF? → It optimises directly on preference pairs, without a reward model and RL.

TL;DR

SFT teaches tasks and formats, preference tuning (RLHF, DPO) teaches which answers people prefer, distillation compresses a big model into a small one. Curate carefully, hold out test data, re-check safety after tuning.

8.4 Why evaluating LLM systems is hard

The problem it solves

Predictive systems (click prediction, delivery times) eventually observe an outcome, even if labels are delayed, partial or biased. Generative systems add a difficulty: **many different outputs can all be acceptable** — "summarise this meeting" has countless good answers, and "was this reply good?" is a judgement, not an observable fact. Yet you still must compare versions, catch **regressions** (things that used to work and now don't) and notice production decay. The engineering problem is turning human judgement into something cheap and consistent that reruns automatically on every change.

How it works: define quality per task

Write down what "good" means first. Some requirements are objectively checkable (valid JSON, compiles, cites a source); others need a **rubric** (written scoring guide) and informed judgement. No label source is perfect, so combine several and investigate disagreements. Chip Huyen's *AI Engineering* (2025) calls

this **evaluation-driven development**: define evaluation criteria before building, as test-driven development writes tests first.

> **TL;DR**
>
> Generative outputs have many right answers, so define task-specific quality criteria and build a repeatable, layered evaluation that runs on every change.

8.5 Offline evaluation: the evaluation pyramid

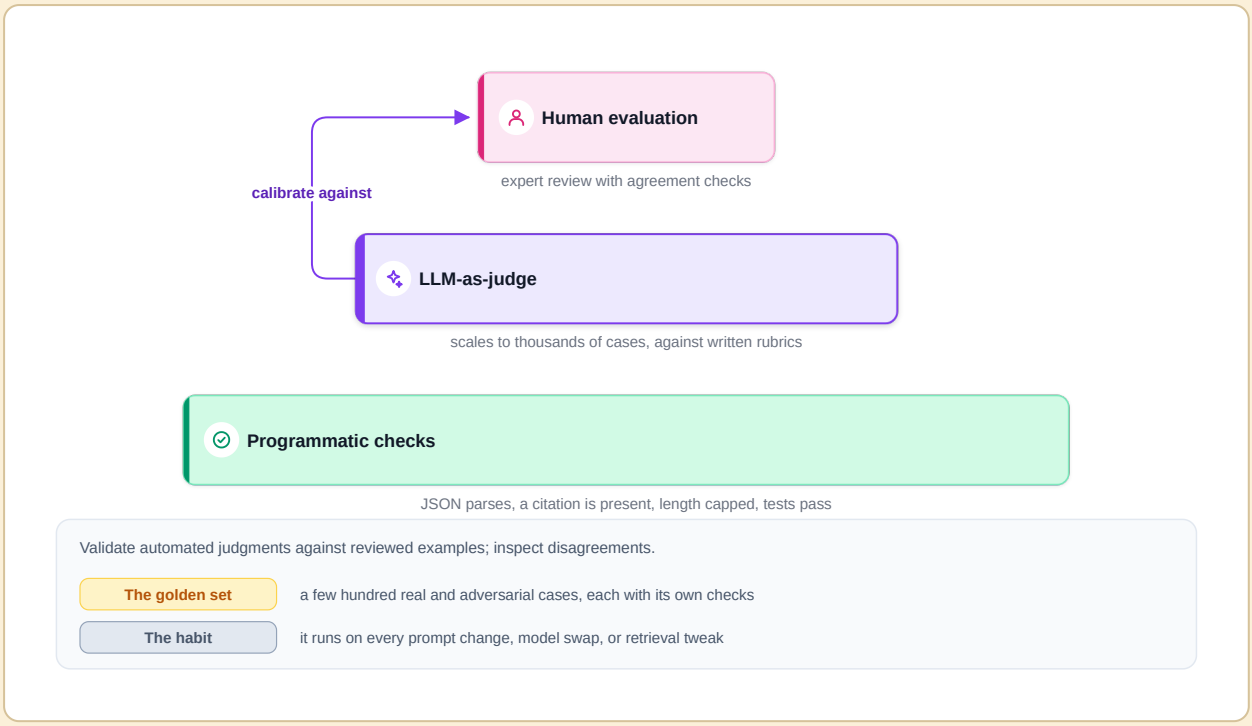


Figure — Three complementary layers, each with limits, run over a golden set (course diagram).

Layer 1 — Programmatic checks

Anything code can assert with no model or human: parses as JSON; required fields present; contains a citation; under a length limit; no banned phrases; generated SQL executes; code compiles and passes unit tests. Cheap, fast, repeatable — the base of the pyramid — but only as strong as what they test: passing tests are *evidence*, not proof, and a present citation needn't support the claim.

Layer 2 — LLM-as-judge

A (usually strong) model grades outputs against a rubric, e.g. *"You will see a user question, the passages we retrieved, and our reply. Rate from 1 to 5 how well the reply sticks to the passages, and name every statement they do not back up."* This layer scales evaluation to thousands of cases. Forms: **single-answer grading**, **pairwise comparison** (which of two is better) and **reference-guided** grading (compare with a gold answer).

Known biases (e.g. Zheng et al., 2023, the MT-Bench / Chatbot Arena paper):

Bias	What happens	Mitigation
Position bias	In pairwise comparisons, favours the answer in a particular position (often the first)	Judge both orders; count inconsistent verdicts as ties
Verbosity bias	Prefers longer answers	Rubric that rewards concision; length-controlled comparisons
Self-preference	Prefers outputs from its own model family or style	Use a different model family as judge; human anchor
Leniency / scale drift	Scores cluster high; shift when the judge model updates	Clear anchored rubric with examples; pin the judge version

A judge must be **validated before it drives release decisions**: compare with a representative human-labelled sample using clear rubrics, and measure agreement. **Cohen's kappa** corrects raw agreement for chance: κ = (p_o − p_e)/(1 − p_e), p_o = observed agreement, p_e = chance agreement. On 100 items labelled "good"/"bad" by judge and human:

	Human: good	Human: bad	Judge total
Judge: good	63	7	70
Judge: bad	7	23	30
Human total	70	30	100

1. p_o = (63 + 23)/100 = 0.86.
2. p_e = 0.7 × 0.7 (both "good" by chance) + 0.3 × 0.3 (both "bad") = 0.49 + 0.09 = 0.58.
3. κ = 0.28/0.42 ≈ **0.67** — "substantial" on the Landis–Koch scale (0.61–0.80). 86% raw agreement shrinks because two raters guessing at a 70/30 split would already agree 58% of the time.

Judge discipline: pin the judge model's version like any dependency; keep a **human-scored anchor set** to recalibrate when the judge changes; mark judge migrations on dashboards, or trend lines become unreadable.

Layer 3 — Human evaluation

Specialists grading samples, side-by-side preference between versions, systematic review of failures. Expensive, slower and fallible — use guidelines, multiple reviewers where it matters, and a disagreement process. Spend it on important samples, new failure categories, high-risk changes and judge calibration.

The golden set

The **golden set** ("eval set", "test set") is a versioned collection of inputs with expected outputs or grading criteria:

1. **Contents**: sampled, anonymised real requests, difficult cases, past incidents, adversarial inputs, and — for RAG — unanswerable questions and permission probes (Chapter 7.11).
2. **Size**: enough items per intent, tricky case and former incident for per-category numbers to mean something, yet cheap enough to rerun constantly; a few hundred is a common start, and rare harms need targeted suites.
3. **Separation**: a stable **held-out** set apart from examples you tune prompts on, or you overfit to your own test.
4. **Growth by accretion**: every representative production failure becomes a test case, like a unit test per fixed bug.
5. **Versioning**: inputs, rubrics, judge and model settings, so comparisons over time are meaningful.

Regression testing: evals as CI

When a prompt is edited, a model replaced or retrieval adjusted, the matching suite runs automatically and can block release — the role unit tests play in **continuous integration (CI)**. Report per category: an average gain can hide a collapse in one category.

Evaluate RAG as two systems

Did authorised, relevant evidence reach the model (retrieval), and did the answer use it correctly (generation)? Evidence arrived → suspect the prompt, chunk selection, the model or output checks; never arrived → indexing, query rewrite or search (Chapter 7.11).

In the interview

> **30-second answer:** "Three layers: code-level assertions, a model grader I've checked against people, and periodic human review, run over a versioned test collection — real traffic, hard cases, attacks, old incidents — whenever prompts, models or retrieval change, the way CI gates code. Human-labelled samples keep the grader honest and reveal gaps. Results are broken down by category, and a held-out slice guards against overfitting."

- *Your judge model updates and all scores shift?* Pin judge versions; recalibrate on the human anchor set; record the migration on dashboards.
- *How big a golden set?* Enough per intent/edge case/incident for meaningful comparisons, cheap enough for every change — commonly hundreds — growing with production failures.

Self-check

1. How do you detect position bias? → Swap the order of the two answers; see if the verdict flips.

TL;DR

Programmatic checks at the base, a calibrated LLM judge in the middle, humans at the top, all over a versioned, growing golden set run like CI.

8.6 Metrics: what each one can and cannot tell you

What it is

Metric	Measures	Good for	Limits
Exact match (EM)	Normalised (lower-cased, punctuation stripped) string equality	Short factual answers, classification	Penalises correct but differently phrased answers
Token F1	Word overlap (precision = share of predicted words that are right; recall = share of reference words found)	Extractive QA	Rewards overlap, not correctness
BLEU (Papineni et al., 2002)	n-gram precision vs references, with a brevity penalty	Machine translation (with several references)	Poor correlation with quality for open-ended generation
ROUGE (Lin, 2004), e.g. ROUGE-L	n-gram / longest-common-subsequence recall/F vs reference	Summarisation benchmarks	Paraphrases score low; fluent wrong text can score high
Embedding similarity (e.g. BERTScore)	Semantic closeness to a reference	Paraphrase-tolerant comparison	Still can't verify facts
pass@k	Probability at least one of k samples passes unit tests	Code generation	Needs good tests; ignores code quality
Faithfulness, answer relevance	Supported by context; addresses the question	RAG	Needs judge or NLI (natural language inference) model (validate it)
Refusal correctness / over-refusal	Abstains when it should, answers when it should	Safety and RAG	Needs unanswerable and benign-but-scary items
Latency, cost per task	Time to first token, total time, $ per completed task	Every system	Must be measured per category, under load

pass@k (Chen et al., 2021, the Codex/HumanEval paper) is estimated without bias by drawing $n \geq k$ samples per problem, counting the c that pass, and computing $\text{pass@k} = 1 - C(n-c, k) / C(n, k)$ ($C(n, k)$ = ways to choose k of n). Worked: $n = 10$, $c = 3$, $k = 3 \rightarrow 1 - C(7,3)/C(10,3) = 1 - 35/120 =$ **0.708**.

Code: a tiny eval harness (runs offline)

`code/eval_harness.py` scores two fake model versions on a nine-item golden set with categories (factual, summary, JSON, unanswerable, guardrail), then runs a regression gate, pass@k, a judge-bias test and a bootstrap confidence interval (a range likely to contain the true difference, estimated by resampling the test items).

```python
def rouge_l(pred, ref):
    """F-score of the longest common subsequence of words."""
    a, b = normalize(pred).split(), normalize(ref).split()
    dp = [[0] * (len(b) + 1) for _ in range(len(a) + 1)]
    for i in range(len(a)):
        for j in range(len(b)):
            dp[i + 1][j + 1] = dp[i][j] + 1 if a[i] == b[j] else max(dp[i][j + 1], dp[i + 1][j])
    lcs = dp[-1][-1]
    ...

def pass_at_k(n, c, k):
    """Unbiased estimator (Chen et al., 2021): 1 - C(n-c, k) / C(n, k)."""
    if n - c < k:
        return 1.0
    return 1 - comb(n - c, k) / comb(n, k)

def check_json(pred, ref):
    """Programmatic check: must parse as JSON (nothing else) and match types/values."""
    try:
        obj = json.loads(pred)
    except json.JSONDecodeError:
        return 0.0
    return float(obj == ref)
```

Output:

```
1) Surface metrics on one reference
   paraphrase, correct  EM=0  F1=0.60  ROUGE-L=0.40
   near-copy, WRONG     EM=0  F1=0.90  ROUGE-L=0.90

2) Golden-set results by category (pass rate)
   category           v1     v2
   factual          0.67   0.67
   summary          0.90   0.60
   json             1.00   0.00
   unanswerable     1.00   0.50
   guardrail        1.00   1.00

3) Regression gate: block if any category drops > 0.10, or any guardrail fails
   v2 BLOCKED by: summary (0.90 -> 0.60), json (1.00 -> 0.00), unanswerable (1.00 -> 0.50)

4) pass@k for a code task: n=10 samples, c=3 pass the unit tests
   pass@1 = 0.300
   pass@3 = 0.708
   pass@5 = 0.917

5) LLM-as-judge position bias on 40 pairwise comparisons
   A wins when shown first : 29/40
   verdict flips when order swapped: 15/40 -> count these as ties

6) Is v2 better overall? Paired bootstrap over the 9 items
   mean v2 - v1 = -0.367, 95% CI [-0.778, +0.078] -> too few items to tell
```

1. **Overlap metrics can invert the truth.** The correct paraphrase ("no data was lost… about twelve minutes") gets ROUGE-L 0.40; a near-copy saying *all data was lost* gets 0.90. Use BLEU/ROUGE for trend-watching on fixed tasks, never as the definition of quality.
2. **Categories hide inside averages.** v2 fixed a factual error (Canberra, not Sydney) but EM marked its *correct* long-form "The transformer paper appeared in 2017." wrong, so factual stays flat; meanwhile it wraps JSON in chatter, returns `"seats": "3"` as a string, and predicts a stock price instead of refusing.
3. **The regression gate** blocks on per-category drops, with a hard rule for guardrails.
4. **pass@k** rises quickly with samples — useful when a system can generate and test several candidates.
5. **Position bias**: the simulated judge favours the first-shown answer when close; swapping flips 15 of 40 verdicts. Judging both orders and treating flips as ties removes the artefact.
6. **Statistics**: with nine items the 95% interval spans zero; small golden sets can't separate close versions.

Common mistakes

- Reporting a single average score.
- ROUGE/BLEU as the main quality metric for open-ended generation.
- Declaring victory from a 2-point change on 50 items with no confidence interval.
- **Benchmark contamination**: public benchmark questions leak into training data, inflating scores; prefer private, product-specific sets.

Self-check

1. Why can ROUGE rank a wrong answer above a right one? → It rewards word overlap, not meaning or truth.
2. A CI spanning zero means? → You can't conclude "v2 is better" from this data.

TL;DR

Choose metrics per category: programmatic checks and pass@k where outputs are verifiable, validated judges for open-ended quality, overlap metrics only as rough signals — always per category with uncertainty.

8.7 Online evaluation: the truth arrives anyway

What it is

Offline evals catch problems before release; production shows real behaviour, with its own biases and noise. Signals from cheap to rich:

1. **Explicit feedback** — thumbs, ratings, comments; a self-selected minority, so sample other sessions too.
2. **Behavioural proxies** — regenerate, rephrase, copy, abandon, escalate to a human. Ambiguous (a regeneration may be exploration; a copy doesn't prove success): validate against task outcomes and reviewed samples.
3. **Task-level outcomes** — the best evidence: ticket resolved, code suggestion accepted and kept (GitHub Copilot popularised acceptance rate), purchase completed. Watch for later reopenings, reverts or harms.

How it works

- **A/B tests** — randomised comparisons decide promotion (randomise users, pre-register the metric, compute confidence intervals).
- **Canary releases** — a small traffic slice on the new version, watched closely before widening.
- **Dashboards** segmented by intent, user tier, topic and language.

Offline and online will sometimes disagree (different users and behaviours) — e.g. the test set favours a new instruction while production regeneration rises. Investigate measurement errors, uncertainty, segments and safety before promoting; a good online proxy never overrides a known safety regression. Add representative failures to the golden set.

In the interview

> **30-second answer:** "Online I track task outcomes where they exist, plus validated behavioural signals — regeneration, abandonment, escalation — and explicit feedback, segmented by intent and user type. A/B tests decide promotion, canaries protect rollouts. When offline and online disagree I investigate rather than trusting one number, and safety regressions block regardless."

- *Negative ratings are flat, yet support complaints keep climbing?* Ratings come only from those who choose to leave them; users who gave up never press a button. Cross-check regeneration and abandonment, review random (not just flagged) samples, then look for a real quality decline.

Self-check

1. Online metric improves but a safety eval regresses — what wins? → Safety: the regression blocks release.

TL;DR

Measure outcomes and validated behaviour in production, promote with A/B tests and canaries, investigate offline/online conflicts, never trade away safety.

8.8 Guardrail evaluations and eval tooling

Guardrail evals

Guardrails keep a system within safety, security and policy limits (input filters, output moderation, refusal behaviour, tool-permission rules). They need their own suites, run on every change and after every fine-tune:

Suite	Tests	Pass condition
Harmful requests	Requests the policy forbids	Refuses (and helpfully redirects where appropriate)
Over-refusal	Benign requests that *sound* risky ("how do I kill a Python process?")	Answers — e.g. the XSTest benchmark measures exaggerated safety
Jailbreaks (tricks to make a model break its rules)	Role-play, obfuscation, multi-turn manipulation	Still refuses
Prompt injection	Instructions hidden in documents, web pages, tool results	Ignores injected instructions; no tool misuse
Data leakage	PII (personally identifiable information), secrets, system prompt, other users' data, permission probes	Nothing leaks
Format and policy	Required disclaimers, banned content, brand rules	Programmatic checks pass

Guardrail failures are usually **hard gates** (any failure blocks — one leak can be an incident regardless of average quality), and adversarial suites need regular refreshing because attacks evolve (red-teaming, Chapter 12).

Eval tooling (concepts, as of 2026)

Need	Examples
Prompt/app test suites in CI	promptfoo, DeepEval, OpenAI Evals
RAG metrics	RAGAS, TruLens
Tracing (recording every step of each request) + eval + datasets platforms	LangSmith, Braintrust, Arize Phoenix, Langfuse, Weights & Biases Weave
Model benchmarking harnesses	EleutherAI lm-evaluation-harness, Stanford HELM, Inspect (UK AI Security Institute)
Crowd-sourced pairwise rankings	LMArena (formerly Chatbot Arena), which ranks models from human votes with a Bradley–Terry / Elo-style (like chess ratings) model

Concepts matter more than brands: versioned datasets, scorers (programmatic, judge, human), experiment tracking, production tracing, and turning traces into new test cases.

Common mistakes

- Testing only that harmful requests are refused, not that benign ones are answered.
- One-time red-teaming instead of a standing schedule.
- Averaging guardrail results with quality results.

TL;DR

Treat safety, injection, leakage and over-refusal as their own suites with hard gates, refreshed continuously; pick tools that version data, trace production and compare experiments.

Chapter TL;DR

- **Adaptation**: prompting is the baseline; **RAG = knowledge** (fresh, private, permissioned, citable); **fine-tuning = behaviour** (format, tone, skill, cheaper small models). Diagnose the observed failure first; combine (RAG + tuned small model) when justified; never use weights as access control; never tune without evals.
- **Training from scratch / continued pretraining**: rarely justified in product design; needs its own data, cost and governance case.
- **LoRA**: $h = W_0x + (\alpha/r)BAx$, B initialised to zero; $r\times(d+k)$ trainable params per matrix (65K vs 16.8M for 4096^2 at r=8); adapters in MBs, mergeable with no latency. Full fine-tuning ≈ 16 bytes/param (~108 GB for 7B); LoRA ~14 GB; **QLoRA** (4-bit NF4 base, double quantisation, paged optimisers) ~4 GB before activations.
- **Data**: SFT (instruction → response), preferences (chosen vs rejected: RLHF, DPO), distillation (teacher → student). Fine-tuning can cause forgetting, safety erosion and more hallucination; re-run all evals after tuning.
- **Evaluation pyramid**: programmatic checks → LLM-as-judge (position, verbosity, self-preference biases; validate with humans, pin versions) → human review; all over a versioned, growing golden set run like CI with per-category gates.
- **Metrics**: EM/F1 for short answers, pass@k = $1 - C(n-c,k)/C(n,k)$ for code, BLEU/ROUGE only as rough signals, faithfulness/relevance/refusal for RAG; report uncertainty.
- **Online**: task outcomes, validated behavioural proxies, feedback; A/B tests, canaries, segmented dashboards; investigate offline/online conflicts; safety regressions always block.
- **Guardrails**: harmful, over-refusal, jailbreak, injection and leakage suites as hard gates, refreshed continuously.